

Name: Varun Kumar L	SRN: PES2UG24C2827
SEC: C	SUB: ML WEEK 14

CNN Image Classification

1.Introduction

The objective of this lab was to build and train a Convolutional Neural Network (CNN) using PyTorch to classify images of hand gestures into three categories: rock, paper, and scissors. The dataset used in this experiment was automatically downloaded from Kaggle and contains real-world images of human hands performing these gestures. The lab involved completing the provided boilerplate notebook, implementing missing model components, training the CNN, evaluating its performance on unseen test data, and analyzing the results. This report summarizes the architecture, training setup, evaluation metrics, and overall findings from the experiment.

2. Model Architecture

The CNN model developed in this lab consists of two major components:

- 1. Convolutional Feature Extractor**
- 2. Fully Connected Classifier**

A. Convolutional Blocks

The convolutional portion of the network contains three sequential Conv-ReLU-MaxPool blocks:

- 1. Block 1**
 - Conv2d: 3 input channels $\rightarrow$ 16 output channels
 - Kernel size: 3×3
 - Padding: 1
 - Activation: ReLU
 - MaxPool2d: 2×2
- 2. Block 2**
 - Conv2d: 16 $\rightarrow$ 32 channels
 - Kernel size: 3, padding=1
 - Activation: ReLU
 - MaxPool2d(2)
- 3. Block 3**
 - Conv2d: 32 $\rightarrow$ 64 channels
 - Kernel size: 3, padding=1
 - Activation: ReLU

B. Fully Connected Classifier

The classifier flattens the feature maps and passes them through two linear layers:

1. Flatten
2. Linear Layer: $64 \times 16 \times 16 \rightarrow 256$
3. ReLU activation
4. Dropout: $p = 0.3$
5. Final Linear Layer: $256 \rightarrow 3$ classes (rock, paper, scissors)

The final layer outputs raw logits, which are used with CrossEntropyLoss for classification.

3. Training and Performance

```
[6] 5m
EPOCHS = 10

for epoch in range(EPOCHS):
    model.train() # Set the model to training mode
    total_loss = 0

    for images, labels in train_loader:
        # Move data to the correct device
        images, labels = images.to(device), labels.to(device)

        optimizer.zero_grad()

        # 2. Perform a forward pass (get model outputs)
        outputs = model(images)

        # 3. Calculate the loss (using criterion)
        loss = criterion(outputs, labels)

        # 4. Perform a backward pass (loss.backward())
        loss.backward()

        # 5. Update the weights (optimizer.step())
        optimizer.step()

    total_loss += loss.item()

    print(f"Epoch {epoch+1}/{EPOCHS}, Loss = {total_loss/len(train_loader):.4f}")

print("Training complete!")
```

Epoch 1/10, Loss = 0.5657
Epoch 2/10, Loss = 0.1570
Epoch 3/10, Loss = 0.0742
Epoch 4/10, Loss = 0.0405
Epoch 5/10, Loss = 0.0326
Epoch 6/10, Loss = 0.0208
Epoch 7/10, Loss = 0.0187
Epoch 8/10, Loss = 0.0134
Epoch 9/10, Loss = 0.0066
Epoch 10/10, Loss = 0.0094
Training complete!

A. Hyperparameters Used

Component	Value
Loss Function	CrossEntropyLoss
Optimizer	Adam
Learning Rate	0.001
Batch Size	32
Total Epochs	10
Train/Test Split	80% / 20%
Device	CPU/GPU (based on availability)

The dataset was transformed using resizing, normalization, and tensor conversion. The training loop included forward pass, loss calculation, backpropagation, and optimizer updates.

B. Final Test Accuracy

```
model.eval() # Set the model to evaluation mode
correct = 0
total = 0

# TODO: Use torch.no_grad()
# We don't need to calculate gradients during evaluation
with torch.no_grad():
    for images, labels in test_loader:
        images, labels = images.to(device), labels.to(device)

        # TODO: Get model predictions
        # 1. Get the raw model outputs (logits)
        outputs = model(images)

        # 2. Get the predicted class (the one with the highest score)
        # Hint: use torch.max(outputs, 1)
        _, predicted = torch.max(outputs, 1) # <-- Replace this

        total += labels.size(0)
        correct += (predicted == labels).sum().item()

print(f"Test Accuracy: {100 * correct / total:.2f}%")
```

Test Accuracy: 98.63%

```

# -----
# 1. Choose any two random classes
# -----

choices = ["rock", "paper", "scissors"]
c1 = random.choice(choices)
c2 = random.choice(choices)

img1_path = pick_random_image(c1)
img2_path = pick_random_image(c2)

print("Randomly selected images:")
print("Image 1:", img1_path)
print("Image 2:", img2_path)

# -----
# 2. Predict their labels using the model
# -----

p1 = predict_image(model, img1_path)
p2 = predict_image(model, img2_path)

print("\nPlayer 1 shows:", p1)
print("Player 2 shows:", p2)

# -----
# 3. Decide the winner
# -----

print("\nRESULT:", rps_winner(p1, p2))

Randomly selected images:
Image 1: /content/dataset/paper/oWI7JL85eXDhRFcV.png
Image 2: /content/dataset/scissors/5mipmm4t8RM5o0tW.png

Player 1 shows: paper
Player 2 shows: scissors

RESULT: Player 2 wins! scissors beats paper

```

4. Conclusion and Analysis

The CNN model performed well on the Rock-Paper-Scissors classification task, successfully learning visual differences between the three hand gestures. The test accuracy indicates strong generalization ability, considering the dataset size and the simplicity of the model.

Challenges Faced

Some challenges encountered during the lab include:

- Ensuring that all transformations matched between training and testing.
- Running the model on CPU made training slower if GPU was unavailable.
- Maintaining the correct shape for the fully connected layer required careful calculation

Possible Improvements

Model performance could be enhanced through:

- Increasing epochs (15–30 epochs often lead to higher accuracy).
- Adding Batch Normalization in convolutional blocks to stabilize gradients.
- Using Data Augmentation (random flips, rotations, lighting changes).
- Expanding model depth, or using pretrained architectures like MobileNet or ResNet.